

DAX / Power Query & Dataverse

Making Power BI Work for Dataverse

Why a Dataverse data model and an analytical data model are not the same thing

Power BI Days DC 2026

Blake Beckemeyer | Lead Software Engineer, Applied Information Sciences



Thank you to our sponsors!



What is Dataverse?

THE SHORT DEFINITION

- Microsoft's managed, cloud-hosted relational database (formerly Common Data Service)
- The shared data layer behind Dynamics 365, Power Apps, and Power Automate
- Fully managed: Microsoft handles infrastructure, scaling, and backups
- You define tables and columns through a UI or API; the platform stores and serves them
- Think “application database with security, logic, and APIs built in”

WHY ANALYSTS RUN INTO IT

- Every Dynamics 365 record — Sales, Service, Field, Marketing — lives in Dataverse
- Custom Power Apps store their operational data there too
- For many organizations it's the source of truth for CRM and operational workflows
- Optimized for transactions — not analytical scans across millions of rows
- If you build Power BI for the business, you'll meet Dataverse fast

Dataverse is optimized for the UI; Power BI is optimized for data aggregation.

How Dataverse works

DATA MODEL & SECURITY

- Data lives in tables (rows + columns); each row has a GUID primary key
- Standard tables (Account, Contact) ship in the box; custom tables extend the model
- Lookup columns create 1:N relationships; N:N is a separate join table
- Choice columns store integer codes plus a label set — not free text
- Role-based security with business units and teams — queries respect what the user can see

HOW YOU QUERY IT

- Power BI / Power Query Dataverse connector — the default starting point
- TDS endpoint: read-only SQL over Dataverse from SSMS or Power BI
- Web API: OData v4 REST endpoint at /api/data/v9.2 for programmatic access
- Link to Microsoft Fabric: shortcut Dataverse tables into OneLake — no ETL, no copies
- All paths honor Microsoft Entra ID auth and the same row-level security

THE CORE IDEA

Two systems, two jobs

Dataverse is built for APPS

- Normalized for transactions and forms
- Polymorphic lookups (a field points to many tables)
- Option sets stored as integer codes
- Timeline is a UX control, not a table
- GUID identity, row-level security, audit

Power BI is built for ANALYTICS

- Star schema: conformed dimensions + facts
- One column, one meaning, one grain
- Labels people recognize, not codes
- Explicit date and event tables to aggregate
- Keys that join cleanly and survive a refresh

If *either* the ERD of Dataverse or the semantic relational model in Power BI is perfect, it breaks the other.

Five beats in sixty minutes

- 1. The happy path** — Connect, load, build a visual — ninety seconds to first chart
- 2. Break it on purpose** — Watch the four places we end up hardcoding or “making it work”
- 3. Fix it in Power BI** — Power Query + DAX patterns for each of the four
- 4. Push it upstream to Fabric** — When and why to solve it once, before Power BI
- 5. Design guidance + gotchas** — Principles you can apply Monday, and the traps to avoid

THE SCENARIO

Your team just built an app... quickly.

- **Scenario:** vibe.powerapps.com might be a data engineer's worst nightmare! Idea to App in 20 mins.
 - The app is a lightweight CRM that helps track touchpoints and updates deals.
- **Opportunity:** Developers don't have their hands directly on the data model in the same way. Set standards early.
- **At least there's some schematic integrity early! And the Plan Designer does love 3NF.**

*The analytic requirements here are put **after** the functional requirements. Not only is this normal, it's the best for the users.*

The four places meaning disappears

01

Polymorphic lookups

One field points to many tables. Customer might be an Account or a Contact.

02

Multi-select option sets

Several values jammed into one cell as a delimited string.

03

Timelines

No timeline table exists — it's a UX object stitched from many activity tables.

04

Keys & identity

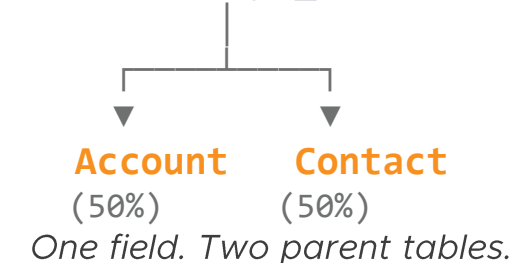
Enforce uniqueness upstream to preserve data refreshes and reduce development frustration.

PROBLEM 1

Polymorphic lookups — “who is the customer?”

- **The Customer field on Opportunity can point to Account OR Contact.**
- The connector gives you `_customerid_value` (a GUID) plus a hidden logical-name hint.
- Join it to Account and half your rows go blank — the Contact ones don't match.
- Join it to Contact and the other half disappear.
- **Neither join is wrong. The model just refuses to pick one table.**

Opportunity._customerid_value



FIX 1

Build a conformed Customer dimension

POWER QUERY · M

```
// Customer dimension = Account U Contact
let
    Source = contact,
    #"Appended Query" = Table.Combine({Source, account}),
    #"Add Customer GUID" = Table.AddColumn(#"Appended
Query", "Customer GUID", each Guid.From([contactid]) ??
Guid.From([accountid]), Guid.Type)
in
    #"Add Customer GUID"
```

PROBLEM 2

Multi-select option sets — many values, one cell

- A **“Service Interests”** field lets users pick several values.
- Dataverse hands you one string: “Cloud; Data; Security”.
- Slice by it and every combination becomes its own fake category.
- “Cloud; Data” and “Data; Cloud” count as two different things.
- **You can't answer “how many accounts want Security?” — the value is trapped.**

One cell:

“Cloud; Data; Security”

▼ explode

account	Interest
A-100	Cloud
A-100	Data
A-100	Security

One row per value.

FIX 2

M Function that Calls the Dataverse API

POWER QUERY · M

```
(entity as text, attribute as text, orgUrl as text) =>
let
    // Build URL
    url =
        orgUrl &
        "/api/data/v9.2/EntityDefinitions(LogicalName='" &
entity & "')/" &
        "Attributes(LogicalName='" & attribute & "')/" &
        "Microsoft.Dynamics.CRM.MultiSelectPicklistAttributeMetadata"
    &
        "?$select=LogicalName&$expand=OptionSet",

    // Call API (inherits Power BI auth)
    Source = Json.Document(Web.Contents(url, [
        Headers = [
            #"Accept"="application/json"
        ]
    ])),

    // Navigate to options
    OptionSet = Source[OptionSet],
    Options = OptionSet[Options],

    // Convert to table
    TableFromList = Table.FromList(Options,
    Splitter.SplitByNothing(), null, null, ExtraValues.Error),
```

```
// Expand fields
Expanded =
    Table.ExpandRecordColumn(
        TableFromList,
        "Column1",
        {"Value", "Label"},
        {"Value", "Label"}
    ),

// Extract label text
Final =
    Table.TransformColumns(
        Expanded,
        {
            {
                "Label",
                each _[UserLocalizedLabel][Label],
                type text
            }
        }
    )
in
    Final
```

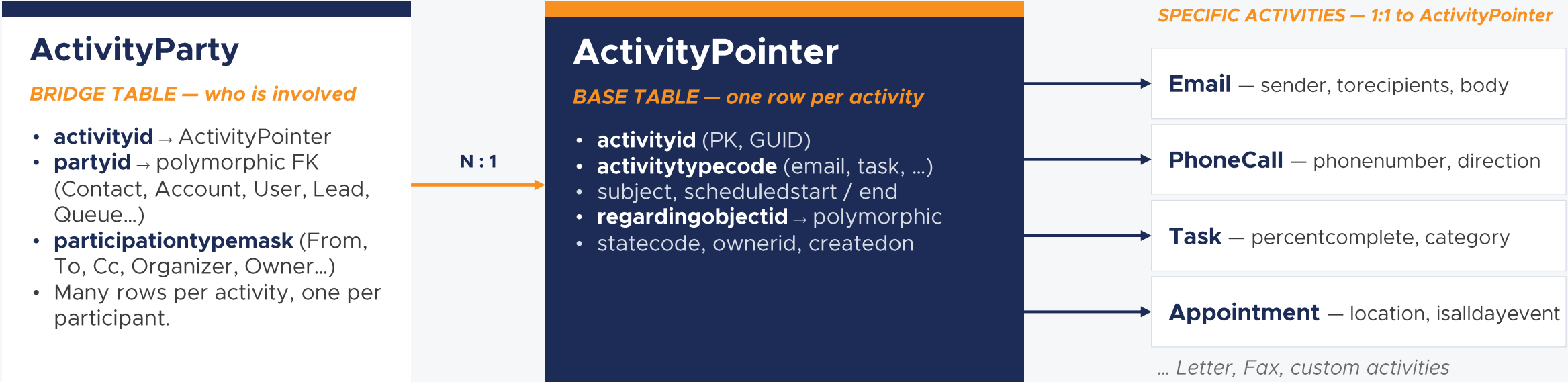
Timelines — the table that isn't there

- ```
email
task] → Timeline
appointment]
```

Or read `activitypointer`  
(the base of all activities).

# Activity tables — ActivityPointer + ActivityParty

One base table for every activity, one bridge table for everyone involved.



**Why it matters for modeling:** activityid is the same GUID across ActivityPointer and the specific table — join on it. ActivityParty is the only path to who participated and how (To / From / Owner). regardingobjectid points to the subject the activity is about; partyid points to the people involved.

### FIX 3

## Use activitypointer for Aggregation, Activity Tables for detail

---

#### DAX

```
Activity Count = COUNTROWS (activitypointer)
Last Touch Date = MAX (activitypointer[createdon])
Days Since Touch = DATEDIFF ([Last Touch Date], TODAY (), DAY)
```

# Alternate keys: a stable handle on every row

- **An alternate key is a uniqueness constraint on one or more business columns (e.g., accountnumber, sku, employeeid).**
- Lives alongside the GUID primary key — same row, stable identity humans and source systems can speak.
- Dataverse enforces it server-side — duplicates fail at write time, not at the next nightly reconciliation.
- *Web API supports Upsert by alternate key — PATCH accounts(accountnumber='ACME-001') with no GUID lookup.*
- **Net effect: a stable, joinable, idempotent key — the missing handle analytics and migrations need.**

account row

*accountid: {A1B2-...}*

alternate key ▼

*accountnumber: ACME-001*

**+ taxid: 12-3456789**

*(uniqueness enforced server-side)*

# Two places alternate keys earn their keep

## MIGRATION · UPSERT

```
PATCH /api/data/v9.2/accounts(accountnumber='ACME-001') // idempotent – re-runnable migration loads, no GUID lookup
```

Same ability in Power Automate.

*Use AltKeys when combining facts from other sources, ex. Historical Data or Financial Data*



# Push it upstream to Fabric

Solve it once at the source, not in every report

# Fabric isn't magic — it's a better place to fix it once

- **Link to Fabric mirrors Dataverse into a Lakehouse / SQL endpoint — no copy job to maintain.**
- Do the conformed dimension, the bridge, the timeline, the key normalization ONCE.
- Gen2 dataflows give you source control and CI/CD for those transforms.
- Every downstream report inherits the clean model — no copy-paste M.
- **Trade-offs: capacity warm-up latency, and it's another platform to govern.**

## Where should the fix live?

### One report, one-off

- Fix in Power Query / DAX

### Many reports, shared model

- Fix in Fabric, once

### Rule of thumb

- Reused = upstream. One-off = in the report.



# Four more “ouch” moments

The traps that bite after the first four are fixed

## BEAT 5

# Gotchas worth a screenshot

---

### #1 Active vs inactive rows

statecode / statuscode aren't applied by default. Unfiltered, deactivated rows inflate every count. Filter to active — or expose state as a real dimension.

### #2 Codes, not labels — and language

Option sets come back as integers. Labels live in stringmap metadata and are language-dependent. Join to the labels, and pin the language.

### #3 Activity Party (email recipients)

To / Cc / From aren't fields on the email — they're rows in activityparty, polymorphic and multi-row. Expect a fan-out; aggregate deliberately.

### #4 Secured columns return nulls

Column / row-level security means refresh sees nulls, not errors, for fields the refresh principal can't read. Silent gaps. Check what the principal can see.

## TAKEAWAYS

# Discussions for you and your AI Business Solutions Architect

---

- 1. Model for the question, not the form** — Reshape the app's model into a star schema before you build visuals.
- 2. Measure downstream development impact**— Convert options to lookups, ensure automations adequately fill all fields, create keys and required fields.
- 3. Make hidden things explicit** — Conformed dimensions, bridges, a real timeline, business keys.
- 4. Fix shared problems upstream** — Reused logic belongs in Fabric; one-offs belong in the report.



# Dataverse isn't bad for BI — it's different

- The connector is fast and the first chart always looks fine.
- Four shapes hide meaning: polymorphic lookups, multi-selects, timelines, keys.
- Each has a clean Power Query + DAX fix — conform, explode, reconstruct, normalize.
- Watch the gotchas: active/inactive, option-set codes, activity party, secured nulls.
- Reused fixes belong upstream in Fabric; one-offs belong in the report.

**Connecting takes ninety seconds. Preserving meaning takes the hour.**



# Questions?

Blake Beckemeyer · Lead Software Engineer · Applied Information Sciences

**[Blake.Beckemeyer@ais.com](mailto:Blake.Beckemeyer@ais.com)**